Task_Manager_Code_Understanding.md

Part 1: Understanding Project Structure

Initial Understanding

The project is a Python-based Task Manager application designed to manage and prioritize tasks. The codebase appears modular, separating models, business logic, and execution logic.

Technologies Identified

Python
Object-oriented programming
File-based data handling
Command-line execution

Main Components

main.py — application entry point
models.py — defines Task, TaskStatus, TaskPriority
scoring/logic files — calculate task importance

Key Insight The application separates

business rules from data models, making it easier to extend functionality.

Part 2: Feature Implementation Planning

To implement a CSV export feature, I searched for areas where task data is processed and stored. The scoring and task management logic indicate that task data is centralized within the task model and related utilities.

Implementation Plan

Create a function to convert task objects into CSV rows

Add export functionality near file handling or task processing logic

Reuse existing task data structures

Affected Components

Task model

Task management utilities

File handling logic

Part 3: Understanding Domain Model

Core Entities

Task — represents work items

TaskStatus — tracks progress (DONE, REVIEW, etc.)

TaskPriority — defines urgency level

Business Logic The system calculates task importance using a scoring algorithm that considers priority, deadlines, tags, and recent updates.

Insight The domain model reflects real-world task management concepts and supports automated prioritization.

Part 4: Practical Application

Files to Modify

Task model or status logic

Business rule/scoring logic

Planned Changes

Calculate how long a task has been overdue

Automatically update status to

“abandoned”
Skip tasks marked as high priority
Questions for Team
Should abandoned tasks be reversible?
Should users receive notifications?
Reflection

AI-assisted prompts helped quickly identify project structure, business logic, and feature locations without reading every line of code. The approach allowed faster onboarding into an unfamiliar codebase.
In the future, I would first identify entry points, domain models, and core business logic before implementing changes.